The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Appendix

B

Enough PHP, SQL and Twig to Read the Shop

Taught in —

Laboratory —

Assessment —

This is a class extract. It is *Appendix B* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter B

Enough PHP, SQL and Twig to Read the Shop

The sibling courses give this appendix to mathematics. This one spends it on language, because that is what this course actually assumes and does not teach.

You are not asked to be a strong PHP programmer. Until Chapter 19 you will read far more code than you write, and this appendix is calibrated to that: enough to follow what the platform is doing, and enough to change it when Chapter 19 asks you to.

B.1 PHP, in Two Pages

B.1.1 The shape of a file

PHP is a language for producing a response to an HTTP request. A file runs top to bottom, every time the page is requested, and whatever it prints becomes the page.

```php
<?php
declare(strict_types=1);

$name  = 'Keyboard';       // string, single quotes: no interpolation
$price = 2499.00;          // float
$qty   = 3;                // int
$total = $price * $qty;

// Double quotes interpolate; single quotes do not. This is the single
// most common source of confusion in a first week of PHP.
echo "Total for $name: $total";   // Total for Keyboard: 7497
echo 'Total for $name';           // Total for $name
```

Three things that surprise everyone

Variables start with `$`, always — including inside strings and in function parameters.

`==` is not `===`. The two-character form converts types before comparing, so `'0' == false` is true and `'abc' == 0` was true in older versions. **Use `===` unless you have a reason not to**, and in Chapter 16 you will see what the other one costs.

`declare(strict_types=1)` is not the default. Without it PHP quietly converts `"5 apples"` into `5` when a function wants an integer. OpenCart's own code does not use it; your modules should.

B.1.2 Arrays, which are really two things

```php
<?php
// A list.
$sizes = ['S', 'M', 'L'];
echo $sizes[0];                    // S
echo count($sizes);                // 3

// A map -- the same type, with keys. Almost every piece of data in
// OpenCart arrives as one of these.
$product = [
    'product_id' => 42,
    'name'       => 'Keyboard',
    'price'      => 2499.00,
];
echo $product['name'];             // Keyboard
```

```php
// Walking one. Note the "as $key => $value" form.
foreach ($product as $field => $value) {
    echo "$field = $value\n";
}

// The ?? operator: use this value, or that one if it is absent.
// Without it, a missing key is a warning and a null.
$image = $product['image'] ?? 'placeholder.png';
```

B.1.3 Functions, classes and the arrow

```php
<?php
function formatPrice(float $amount, string $symbol = 'Rs. '): string
{
    return $symbol . number_format($amount, 2);
}

class ShippingQuote
{
    private float $rate;

    public function __construct(float $rate)
    {
        $this->rate = $rate;          // $this-> reaches a property
    }

    public function forWeight(float $kg): float
    {
        return $this->rate * max(1.0, $kg);
    }
}

$q = new ShippingQuote(120.0);
echo formatPrice($q->forWeight(2.5));    // Rs. 300.00
```

The three arrows, which are different

PHP uses three symbols that beginners merge into one.

- `$obj->method()` — call a method on an object, or read a property. An *object*.
- `$array['key']` — read a key from an array. An *array*. Square brackets, quoted key.
- `Class::CONSTANT` — reach something on the class itself rather than on an instance.

OpenCart hands you arrays far more often than objects, so `['key']` is the one you will type most. `$this->request->get['product_id']` is all three ideas in one expression: a property of a property, then an array key.

B.1.4 Where request data arrives, and why it is dangerous

```php
<?php
// Submitted by a form with method="get"  -> $_GET
// Submitted by a form with method="post" -> $_POST
$qty = $_POST['quantity'] ?? 1;

// EVERY value in those arrays is a STRING, and every one of them was
// typed by somebody who may not be a customer. Chapter 16 is about
// what happens when this is forgotten; Chapter 15 is about sessions,
// which is how the shop remembers who you are between requests.
$qty = (int)$qty;                    // now it is certainly an integer
```

```php
if ($qty < 1 || $qty > 999) {
    $qty = 1;                          // and certainly in range
}
```

B.2 SQL, as This Course Uses It

You met SQL in *Database Systems*. This section is a reminder of the five statements this course needs and the two ideas it leans on.

```sql
-- Read. The only one you will type often.
SELECT p.product_id, pd.name, p.price
  FROM oc_product p
  JOIN oc_product_description pd ON pd.product_id = p.product_id
 WHERE pd.language_id = 1
   AND p.status = 1
 ORDER BY p.price DESC
 LIMIT 20;

-- Count. Chapter 6 shows why a category page runs one of these as
-- well as the query above, every single time.
SELECT COUNT(*) FROM oc_product WHERE status = 1;

-- Change one row. The WHERE is not optional; without it you have
-- just repriced the entire catalogue.
UPDATE oc_product SET price = 2999.00 WHERE product_id = 42;

-- Remove. Same warning, more so.
DELETE FROM oc_product_to_category WHERE product_id = 42;

-- Make a query fast, or fail to -- see Chapter 6.
CREATE INDEX idx_sort_order ON oc_product (sort_order);
```

The two ideas everything else rests on

A join combines rows from two tables on a matching column. OpenCart splits a product across eighteen tables (Chapter 6), so almost every useful question needs at least one join.

An index is a sorted structure the database can search instead of reading every row. It makes reads faster and writes slower, and it is useless if the query hides the column inside a function — `ORDER BY LCASE(name)` cannot use an index on `name`, which is the central finding of Chapter 6.

EXPLAIN, which answers "why is this slow?"

Put `EXPLAIN` in front of any `SELECT` and the database tells you what it intends to do rather than doing it. Three words in the output carry nearly all the information:

- **`type: ALL`** — a full table scan. Every row read. At fifty thousand products this is the one to fix.
- **`Using filesort`** — the result had to be sorted after it was fetched, because no index gave it in the right order.
- **`Using temporary`** — a temporary table was built to hold intermediate results, usually because of `DISTINCT` or `GROUP BY`.

Chapter 6 runs `EXPLAIN` against the real category query and finds all three at once.

B.3 Twig, Which Is Just the Page

OpenCart's templates are Twig. A template is HTML with three kinds of hole in it, and that is very nearly the whole language.

```
{# A comment. It does not reach the browser. #}

{# 1. Print something. #}
<h1>{{ heading_title }}</h1>
<span class="price">{{ product.price }}</span>

{# 2. Do something -- a loop or a condition. #}
{% if products %}
  <ul>
    {% for product in products %}
      <li>
        <a href="{{ product.href }}">{{ product.name }}</a>
        {% if product.special %}
          <s>{{ product.price }}</s> {{ product.special }}
        {% else %}
          {{ product.price }}
        {% endif %}
      </li>
    {% endfor %}
  </ul>
{% else %}
  <p>{{ text_empty }}</p>
{% endif %}

{# 3. Filter -- transform a value on the way out. #}
<p>{{ product.description|striptags|slice(0, 120) }}</p>
<p>{{ product.name|escape }}</p>
```

|escape and what it prevents

Twig escapes output by default in most configurations, and that default is the single thing standing between a product description written by somebody else and a cross-site scripting hole in your shop.

Twig also offers `|raw`, which turns the escaping off. There are legitimate uses, and every one of them is a decision to trust the source of that value completely. Chapter 16 looks for places where somebody has used it without meaning to.

B.4 YAML, in Ten Lines

The Compose file is YAML, and YAML has exactly two rules worth knowing.

```
# 1. Indentation is structure. Two spaces per level, NEVER tabs --
#    a tab is a syntax error and the message will not say so clearly.
services:
  db:
    image: mariadb:11
    environment:
      MARIADB_DATABASE: opencart

# 2. A leading dash is a list item.
    ports:
      - "8090:80"
      - "3306:3306"
```

That is enough to read `code/docker-compose.yml` and to change the one thing Chapter 5 asks you to change, which is a port number.

B.5 Where To Look Next

For	Go to
PHP	`php.net/manual` — the official manual, and unusually good. Each function page has user-contributed notes; treat those as hints, not authority
SQL	The MariaDB knowledge base, and your *Database Systems* notes for the theory this appendix skips
Twig	`twig.symfony.com/doc` — the "for template designers" page is all you need
OpenCart	`docs.opencart.com`, and the source in your own container, which is more current than the documentation

Read the source; it is right there

The shop's entire source is inside your container, and you can read it:

```
docker compose exec shop \
  cat /var/www/html/catalog/model/catalog/product.php
```

This is not a last resort. Chapter 6 finds the category page's real query this way, and it is the only way to answer "what does this page actually do?" with certainty rather than with a plausible guess. The documentation describes a version; the container holds yours.